Python Data Manipulation Problem Assignment

Intern: Sumit Dhar
Date: 26th September, 2025

Objective:

To design and implement a modular Python-based data manipulation system that loads structured JSON data into a database, applies transformations (email formatting, postal code cleaning, and phone number encoding), and outputs a clean and consistent dataset for further processing or analysis.

Creating a strong MVP:

- **LOAD** → JSON file ingested into SQLite database.
- **TRANSFORM** → Apply modular services for:
  - *Email Formatter* (normalize addresses)
  - *Postal Cleaner* (standardize postal codes)
  - *Phone Encoder* (convert to ASCII-based encoding)
- **STORE** → Persist the cleaned data back into the database.
- **VERIFY** → Retrieve and display transformed records for validation.
- **Structure** → Follows modular and scalable design (separate service files).
- **Covers** → Data ingestion (Extract), transformation (Transform), and persistence (Load) – i.e., a full ETL cycle.

Step 1: Created the folder structure:

```
data_manipulation_problem/
├── run.py
├── data/
│   └── sample_data_for_assignment.json
├── db/
│   ├── __init__.py
│   ├── connection.py
│   ├── fetcher.py
│   ├── loader.py
│   └── json_to_sql.db
├── services/
│   ├── __init__.py
│   ├── email_formatter.py
│   ├── postal_cleaner.py
│   └── phone_encoder.py
└── requirements.txt
```

Step 2: Solved all the questions:

Question 1: Unload the data from MySQL to pandas dataframe.

Here we have used SQLite and SQLAlchemy.

```
# database/fetcher.py
# fetches table → pandas, from SQLite database using SQLAlchemy

import pandas as pd
from db.connection import get_engine

def fetch_table_as_df(table_name: str = "json_to_sql_table") -> pd.DataFrame:
    engine = get_engine()
    query = f"SELECT * FROM {table_name}"
    df = pd.read_sql(query, engine)
    return df
            col = col.strip()
            if col:
                parts = col.split(None, 1)  # split into name + type
                if len(parts) == 2:
```

Question 2: Write a program to display all data under their respective column name as a pandas dataframe.

Here we have used SQLite and SQLAlchemy.

```
# Step 2: Fetch back as pandas DataFrame

    table_name = "json_to_sql_table"

    df = fetch_table_as_df(table_name)

    print("\nDataFrame from DB before formatting:")

    print(df.head())  # shows first 5 rows
```

```
DataFrame from DB before formatting:
   id     name              email postalZip           phone
0   1    Alice  alice@example.com     12345  (816) 530-4269
1   2      Bob        bob@xyz.com     67890  1-811-920-9732
2   3  Charlie    charlie@abc.org    A12B3C  (123) 456-7890
3   4    David   david@domain.net     54321    987-654-3210
4   5      Eve     eve@sample.com     12X45  (555) 123-4567
```

Question 3: Write a program to change all email address to have the format – **abc@gmail.com** ' (note the ending should be gmail.com).

Here we have used SQLite and SQLAlchemy.

```python
# services/email_format.py

import pandas as pd

def format_email(df: pd.DataFrame, column: str = "email") -> pd.DataFrame:
    """
    Convert all emails to format: username@gmail.com
    """
    if column not in df.columns:
        raise ValueError(f"Column '{column}' not found in DataFrame")

    df[column] = df[column].apply(lambda x: f"{str(x).split('@')[0]}@gmail.com")
    return df
```

Question 4: The values for "postalZip" doesn't have same type of data (some are string, some are int, some are alpha numeric). Write a program to convert all values to int. In case of alpha numeric remove the letters and keep only the numbers and convert it to int.

Here we have used SQLite and SQLAlchemy.

```python
# services/postalZip_format.py

import pandas as pd
import re

def clean_postal_zip(df: pd.DataFrame, column: str = "postalZip") -> pd.DataFrame:
    """
    Convert all postalZip values to integers.
    Remove letters, keep only numbers.
    """
    if column not in df.columns:
        raise ValueError(f"Column '{column}' not found in DataFrame")

    def to_int(val):
        # Convert to string, remove non-digit characters
        s = str(val)
        digits = re.findall(r'\d+', s)
        if digits:
            return int("".join(digits))
        else:
            return 0

    df[column] = df[column].apply(to_int)
    return df
```

Question 5: Write a separate function which take input data as values under the column name "phone" and processes it to return equivalent ASCII char of the value taken 2 at a time from left to right. Considering the value should be in between 65 to 99. If two-digit number is less than 65 replace it with O (O for Orange).

Here we have used SQLite and SQLAlchemy.

```python
import pandas as pd
import re

def format_phone_number(df: pd.DataFrame, column: str = "phone") -> pd.DataFrame:
    if column not in df.columns:
        raise ValueError(f"Column '{column}' not found in DataFrame")

    def phone_number_to_ascii(phone):
        # Remove non-digit chars
        digits = re.sub(r'\D', '', str(phone))
        result = ""
        for i in range(0, len(digits) - 1, 2):  # take 2 digits at a time
            num = int(digits[i:i+2])
            if 65 <= num <= 99:
                result += chr(num)
            else:
                result += "O"
        return result

    new_column = "coded_phone_number"
    df[new_column] = df[column].apply(phone_number_to_ascii)
    df.drop(columns=[column], inplace=True)
    return df
```

Step 2: Run **run.py** to complete all the processes.

`python run.py`

```
(testing_DG_1) PS F:\Internship\Python\data_manipulation_problem> python run.py
Data loaded into table: json_to_sql_table

DataFrame from DB before formatting:
   id     name             email postalZip          phone
0   1    Alice  alice@example.com     12345  (816) 530-4269
1   2      Bob        bob@xyz.com     67890  1-811-920-9732
2   3  Charlie    charlie@abc.org    A12B3C  (123) 456-7890
3   4    David   david@domain.net     54321    987-654-3210
4   5      Eve     eve@sample.com     12X45  (555) 123-4567

DataFrame from DB after formatting:
   id     name             email  postalZip coded_phone_number
0   1    Alice    alice@gmail.com      12345              QAOOE
1   2      Bob      bob@gmail.com      67890              OO\OI
2   3  Charlie  charlie@gmail.com        123              OOONZ
3   4    David    david@gmail.com      54321              bLOOO
4   5      Eve      eve@gmail.com       1245              OOOOC
(testing_DG_1) PS F:\Internship\Python\data_manipulation_problem>
```

Summary:

This project demonstrates a professional ETL-style pipeline with a clear separation of concerns:

- **Data Loading**: JSON input is ingested and stored in a SQL database.
- **Services Layer**: Independent, reusable transformation modules handle specific tasks:
  - ***Email Formatter***: Normalizes all email addresses to Gmail format.
  - ***Postal Formatter***: Standardizes postal codes into integer values by removing non-numeric characters.
  - ***Phone Formatter***: Encodes phone numbers into character sequences based on two-digit ASCII mapping.
- **Execution Pipeline**: The transformations are executed sequentially through a single entry point (run.py), ensuring clean, maintainable, and scalable data manipulation.

This modular structure reflects **industry practices** in ETL and data engineering, making the system easy to extend for additional transformations or integration with larger data workflows.
